MFLocator: 基于多源特征融合的开源软件漏洞定位到补丁方法

谢晓园，宋壹，许淋，谢东辰，周纯英，张赫

(武汉大学 计算机学院, 湖北 武汉 430072)

摘 要: 随着开源软件的广泛应用，开源软件漏洞对软件系统安全造成的隐患也日趋严重。补丁是修复漏洞的通用方式，然而，有相当一部分已经开发完毕的补丁被直接提交到开源代码仓库，没有与对应的漏洞建立显式链接，导致受该漏洞影响的下游软件无法得到及时修复，极大危害软件系统的安全。由此可见，根据漏洞准确定位到对应的补丁，对于确保软件安全具有重要意义。针对这一任务，现有方法通常使用基于匹配或排序的策略，但由于对漏洞描述和提交信息描述中的语义特征挖掘仍然不够深入充分等问题，存在定位不够精确的缺点。为此，实现一种基于多源特征融合的开源软件漏洞定位到补丁方法 MFLocator (**M**ulti-**F**eature Vulnerability Patch **Locator)**: 首先定义 37 种能够反映漏洞与补丁之间相关性的手工特征，然后将漏洞描述和提交信息描述成对输入到预训练模型得到语义特征，再将两种特征进行拼接，输入到更适用于不平衡数据集的三种分类模型 (XGBoost、LigthGBM、CatBoost)中，综合三者的预测结果得到最终排序。使用来自 10 个开源软件项目的 1669 个漏洞进行评估，结果表明，MFLocator 能够达到的 Recall@*K*、Manual Efforts@*K* 和 NDCG@*K* 指标值均优于该领域现有的代表性方法，能够有效地进行漏洞补丁定位。

关键词: 开源软件漏洞; 补丁定位; 语义特征; 预训练模型; 软件安全

中图法分类号: TP311

1 绪 论

近年来，软件行业蓬勃发展，软件规模日益扩大的同时，软件的结构与内部逻辑也愈发复杂。软件开发中复杂的开发逻辑和庞大的开发工作量，使得开发者们难以独自完成整个项目[1]。而开源软件 (Open Source Software, OSS)的出现有效缓解了此类问题。因为 OSS 的存在，开发者无需独自开发整个软件项目，而是可以将 OSS 作为组件整合到自己的软件中，或者以 OSS 为基础框架来搭建自己的软件，从而减少工作量、提高工作效率[2]。正因上述优点，OSS 被广泛应用到软件开发中。然而，OSS 的大规模应用也带来了巨大的隐患：一旦某个 OSS 存在漏洞 (Vulnerability)，使用其作为组件或以其作为框架进行开发的软件系统都会承担与之相同的风险[3]。

随着 OSS 的发展，在 OSS 中发现的漏洞数量也在不断增加。为了推动对漏洞的披露和修复工作，通用漏洞披露 (Common Vulnerabilities and Exposures, CVE)[4] 和美国国家漏洞数据库 (National Vulnerability Database, NVD)[5]等公共安全平台建立了有效机制，向公众提供已披露漏洞的详细信息及其相关补丁 (Patch)的链接[6]。尽管这使得对漏洞的管理更加科学高效，但现有研究表明，仍有近 57%的 CVE 漏洞缺乏指向补丁的链接；同样地，在 OSS 代码存储库已有的历史提交 (Commit)中，只有 12%引用了相应的 CVE ID[7]。这意味着，即使有些漏洞已被修复，平台的维护者 (例如 CVE 编号机构，CNAs)可能也不会更新 CVE/NVD 上的补丁链接，这使得 OSS 存在的漏洞难以得到机会修复，更使得集成该 OSS 的软件系统开发者无法及时得到通知，进而严重威胁软件系统的质量和安全，产生“连锁反应”。例如，Log4J 这一最受欢迎的 Java 库之一，就曾受到 Log4Shell (CVE-2021-44228)漏洞严重的攻击威胁。Log4Shell 以 Log4J 为目标，通过记录指向可能不受信任的 Java 类的网址，诱骗客户端应用程序执行恶意代码。大量使用 Log4J 的应用程序和云服务，包括

Steam、Apple iCloud 和 Minecraft 在内，都受到该漏洞的影响[8]。此外，中国网络安全头部企业奇安信在其于 2024 年 8 月最新发布的《中国软件供应链安全分析报告》中指出，国内企业的软件项目几乎 100%使用了开源软件，而这些项目中 88%含有开源软件漏洞，其中大部分是超危或高危漏洞[9]。由此可见，定位到漏洞所对应的安全补丁是一项必要且迫切的任务，其对于修复漏洞、维护 OSS 的质量乃至确保软件系统的安全，都有着十分重要的意义。

OSS 漏洞补丁定位是根据 CVE、NVD 等漏洞披露平台披露的漏洞信息，在 GitHub 等 OSS 代码存储库的历史提交中寻找对应安全补丁的过程。在早期的工作中，漏洞补丁定位通常是手动完成，虽然定位的准确率很高，但是效率极低，很难完成大规模的补丁定位工作[13]。因此，研究人员尝试使用漏洞数据库中的辅助信息来定位补丁，如历史提交中的 CVE ID 和 CVE/NVD 页面中的超链接[10]。与手动定位相比，这种基于文本匹配的定位方法在一定程度上提升了工作效率，但由于忽略了文本中的语义信息，很难保证补丁的精确定位[11]。为缓解这一问题，现有工作尝试基于排序的补丁定位，即从漏洞描述信息和提交中手动定义并提取特征，如漏洞描述信息和提交之间的共享单词数，并使用生成的特征向量训练排序模型或分类器[7, 12]。与基于匹配的补丁定位方法相比，基于排序的方法虽然能够在实现快速定位的同时提高其准确性，但仍存在对漏洞描述和提交信息描述中的语义特征挖掘不够深入充分等问题，直接影响漏洞补丁定位任务的有效性。

为此，实现基于多源特征融合的开源软件漏洞定位到补丁方法 MFLocator (**M**ulti-**F**eature Vulnerability Patch **Locator)**。具体而言，首先定义 37 类手工特征，捕捉漏洞信息和提交信息之间的异同；然后将漏洞描述和提交中的自然语言部分拼接并输入到基于 BERT[15]的预训练模型 RoBERTa[14]中，得到相应的语义特征向量，实现成对提取漏洞描述和提交信息描述中的语义信息，并在该过程中使用经过负采样的训练集对 RoBERTa 模型进行微调；在排序综合阶段，将手工特征与语义特征向量分别输入进用三个适用于不平衡样本的分类模型 XGBoost、LightGBM 和 CatBoost[16]；最后，综合三个分类模型的结果做出最终预测[12]，实现基于排序的漏洞补丁定位。

使用 10 个开源项目中的 1669 个漏洞[12]来评估 MFLocator 的性能，实验结果表明，该方法的 Recall@K、Manual Efforts@K 和 NDCG@K 指标值均优于该领域现有的代表性技术，显示其能够有效识别漏洞的安全补丁，实现更好的漏洞补丁定位效果。

2 背景知识

2.1 NLTK库

NLTK (Natural Language Toolkit)是由宾夕法尼亚大学计算机和信息科学系的 Steven Bird 和 Edward Loper 开发的 Python 库[22]，广泛应用于自然语言处理领域。它提供了分词、停用词过滤、词干提取和词形还原等数据预处理方法，支持词性标注、命名实体识别、语义分析和关系抽取等多种自然语言处理任务。NLTK 为 50 多种语料库和词汇资源 (如 WordNet)提供易于使用的接口，成为用户学习和处理自然语言的重要工具。在漏洞定位到补丁任务中，常用于对漏洞描述和提交信息进行数据预处理。

2.2 TF-IDF

TF-IDF 即词频-逆文档频率，它从文档词频和文档库中词的稀有程度两方面来评估词的重要程度，是一种常用于信息检索和数据挖掘的技术[23]。其中，TF 是词在文档中出现的频率与文档总词数的比值；IDF 是文档库中文档总数与出现该词的文档数量的比值，如公式(1)所示。如果一个词在多数文档中出现，表示该词普遍存在，重要性较低，IDF 值较小。TF-IDF 的计算如公式(2)所示，其中TF-IDF(t, d, D)表示在文档库D里，文档 d 中 token t 的 TF-IDF 值，DF(t)表示出现词 t 的文档数目。可用 TF-IDF 向量表示文档，向量的每个维度对应词汇表中的一个词在该篇文档中的 TF-IDF 值。在漏洞定位到补丁任务中，常用 TF-IDF 向量的余弦相似度判断漏洞和提交描述文本的相似性。

$$\mathrm{IDF}(t, D) = \log \frac{|D|}{\mathrm{DF}(t) + 1}, \tag{1}$$

$$\text{TF-IDF}(t, d, D) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t, D). \tag{2}$$

2.3 预训练模型

预训练模型首先使用大量数据进行训练，而后在初步训练好的模型上进行继续训练或其他操作。预训练模型在进行最终的预测前包括两个阶段：预训练和微调。在预训练阶段，通常会使用大规模的语料信息并采用无监督或弱监督的方式训练模型，以使模型能够获得语言相关的知识，如句法、语法等；在微调阶段，可以直接使用预训练好的模型，再根据需要进行定制化地训练，使得模型能够更擅长处理某些特定的任务，如文本分类、序列标注等。本作品的实验部分使用 BERT 预训练模型的变体 RoBERTa 模型来完成对自然语言对(的语义信息提取。

首先介绍 BERT 预训练模型。BERT 基于主流的 Transformer 架构，具有可变数量的编码器层和自注意力头，并可以根据给定的方案处理输入的单句或句子对。BERT 通过无监督学习在大型语料库上预训练双向词汇表征和丰富语义知识，以达到两个目标，一是预测句子中的缺失词 (即掩码语言模型)，二是理解句子之间关系及下一句预测 (Next Sentence Prediction，NSP)。在实际使用中，可以在较小的数据集上使用较少的资源对预训练模型 BERT 进行微调，以优化其在特定的自然语言处理任务 (如摘要、问答、机器翻译、情感分析等)上的表现。最后，BERT 输出与输入序列中的 token 相对应的隐藏状态向量。

为了适应不同的训练目标，通过对 BERT 进行改进，衍生出许多新的预训练模型，如 RoBERTa。RoBERTa 模型同样基于 Transformer 架构，且具有与 BERT 相同的输入和输出格式。其删除了 BERT 中的 NSP 任务，并将掩码策略从静态修改为动态。经过上述改进，RoBERTa 在处理自然语言对上表现出良好的性能。

2.4 预测模型

对于每个给定的漏洞，只有少数提交是其安全补丁 (即正样本)，而其余提交可能是与漏洞修复无关的功能更新等 (即负样本)，因此定位安全补丁是针对极度不平衡数据的分类问题。传统的机器学习模型在这种情况下非常容易出现欠拟合，导致无法准确预测[20]。当前，主要有三种在不平衡数据上表现较好的分类模型：XGBoost、LightBGM 和 CatBoost。

- XGBoost (Extreme Gradient Boosting)

  该模型采用“树的叠加”技术构建模型，通过叠加多个弱分类器以实现强分类的效果[17]。XGBoost 算法的核心思想是通过构建一系列有序的决策树逐步改进模型。具体而言，模型以顺序的方式逐步添加弱分类器，不断减小前一分类器的残差，从而使最终的预测结果更接近真实值。对于每个样本而言，模型的预测结果是所有决策树对应值的总和。XGBoost 不仅在准确性上表现优异，还具备高效的计算性能和良好的可扩展性，因此在分类、回归、排名等任务中得到了广泛应用。此外，其正则化机制有效防止了过拟合，使得模型在处理复杂数据时更加稳健。

- LightGBM (Light Gradient Boosting Machine)

  该模型基于直方图的决策树算法，通过特征离散化处理，相比于 XGBoost 遍历所有特征值的方式显著节省了时间和空间开销[18]。在决策树的生长策略上，LightGBM 采用按叶生长 (leaf-wise)的方式，可以根据大规模学习数据排除大部分小梯度样本，在减小数据量与保持高精度之间实现良好平衡。LightGBM 是一种快速高效、低内存占用、高准确度，并支持并行和大规模数据处理的算法。

- CatBoost (Categorical Boosting)

  该模型采用完全对称树的构建方式，结构相同的决策树提高了模型的稳定性和可解释性[19]。与 XGBoost 和 LightGBM 不同，CatBoost 不需要对分类特征进行预处理，而是直接在训练时通过一种称为“目标编码”的方法，使用类别特征的目标变量统计信息 (如均值)来代替类别特征的值。它在保持数据泄漏风险最小化的同时，有效地利用了类别特征的信息。此外，CatBoost 采用排序提升的方法对抗训练集中的噪声点，避免梯度估计的偏差，进而解决预测偏移的问题。CatBoost 具备良好的计算效率和可扩展性，能够高效合理地处理类别型特征，大幅降低过拟合的可能，具有高精确度和高泛化能力，在需要处理大量类别特征的场景中表现优异。

3 方 法

3.1 概述

安全补丁是由 OSS 开发人员开发或部署在其代码存储库中的对漏洞进行修复的提交[7]。对于给定的漏洞和提交集合，根据提交与漏洞信息之间的相关性对所有提交进行排序，使得安全补丁的排名尽可能高，帮助开发人员更快地利用安全补丁对漏洞进行修复。

漏洞补丁定位技术的输入是漏洞和提交的相关信息：漏洞集合$V=\{v_1,v_2,\ldots,v_{|V|}\}$中的每个漏洞$v_i$分别对应一个提交集合$C_i=\{c_1,c_2,\ldots,c_{|C_i|}\}$。其中$v_i$是由漏洞的 CVE ID *cve_id*,、描述信息 *desc*、CWE 类型 *cwe*、CVE ID 分配时间 *time* 以及 NVD 页面中的超链接 *hyperlinks* 组成的元组，即$v_i=(cve_id_i, desc_i, cwe_i, time_i, hyperlinks_i)$；$c_j$是由提交哈希值 *commit_id*、描述 *msg*、与上一提交的差异代码 *diff_code* 以及提交时间 *commit_time* 组成的元组，即 $c_j = (commit_id_j, msg_j, diff_code_j, commit_time_j)$。输出是$C_i$中各个提交的排名$R_i = [r_1, r_2, \ldots, r_{|C_i|}]$，其中$r_j$为提交$c_j$与漏洞$v_i$的相关性排名，排名越靠前表示提交是漏洞对应安全补丁的可能性越大。

围绕上述任务，实现 MFLocator 方法，图 1 展示了该方法的工作流，它由数据预处理、手工特征生成、语义特征生成、特征拼接和排序综合五个阶段组成。

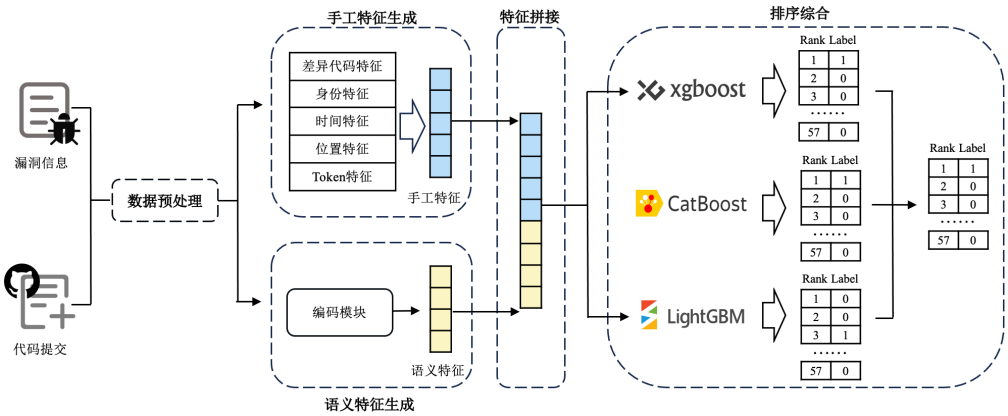

图 1 MFLocator 漏洞补丁定位工作流

其中，数据预处理阶段对数据集进行预处理操作，例如分词、去停用词等；手工特征生成阶段从不同维度提取体现漏洞和提交相关性的手工特征，包括差异代码、身份、时间、位置、Token 等五类共 37 个特征；语义特征生成阶段使用 RoBERTa 预训练模型将漏洞描述和提交信息描述对转化成语义特征向量；特征拼接阶段对上述提取的手工特征和语义特征进行融合；排序综合阶段使用手工特征和语义特征对三个分类模型进行训练，并在部署时综合三组排序得到最终结果。下面分别对这五个部分进行介绍。

3.2 数据预处理

将漏洞描述和提交的描述、差异代码进行分词，为方法的手工特征生成阶段做准备。具体而言，参考 Wang 等人使用的分词方法定义一组状态转换规则，若当前字符与下一字符满足某一状态转换规则，则进行字符串切分，得到 token (具体细节请查阅参考文献[12])。据此分别得到漏洞词袋和提交词袋，取两者交集作为数据集中有效 token 的词袋。后续使用漏洞描述和提交文本计算特征时，将忽略不在该词袋中的 token。

使用大小写统一化、分词、去停用词和词干提取等常用的自然语言文本预处理方法来简化文本，有效提取文本中有用的信息，为方法的语义特征生成部分做准备。具体而言，使用 Python 中面向字符串的 lower()方法将文本中的大写转换为小写；使用 RobertaTokenizer 进行分词，便于后续对单词进行处理；使用 NLTK 库中的英文停用词包去除 “the”、“is”等在自然语言中频繁出现但语义信息较少的单词；使用 NLTK 库中的 SnowballStemmer 去除单词的前后缀得到词根，将不同词性、时态或单复数形式的同一词汇映射为统一的拼写方式，减少同一词汇的形式多样性。

3.3 手工特征生成

对给定的漏洞和提交对，提取差异代码、身份、时间、位置、Token 五个维度的手工特征，如表 1 所示。

表 1 MFLocator 手工特征

维度	特征	含义
差异代码特征	Added Line Num	提交中添加代码的行数
	Deleted Line Num	提交中删除代码的行数
	Modified Line Num	代码提交中修改代码的行数
身份特征	CVE Num	提交信息描述中提到的 CVE 数量
	Bug Num	提交信息描述中提到的 Bug 数量
	Issue Num	提交信息描述中提到的 Issue 数量
	URL Num	提交信息描述中提到的 URL 数量
	CVE Match	提交信息描述中提到的 CVE ID 与漏洞的 CVE ID 是否一致
	Bug Match	提交信息描述中提到的 Bug ID 与漏洞相关的 Bug ID 是否一致
	Type Relevance	提交与漏洞在漏洞类型方面存在相关性的数量
	Impact Relevance 1	提交提及的漏洞类型与漏洞的影响存在因果关系的数量
	Impact Relevance 2	漏洞的类型与提交提及的漏洞影响存在因果关系的数量
时间特征	Time Interval	提交时间与漏洞 CVE ID 分配时间的间隔
位置特征	Same Filepath Num	提交与漏洞描述中相同文件路径的数量
	Same Filepath Ratio	相同文件路径的数量与提交中修改的文件总数的比值
	Unrelated FilePath Num	在提交中出现但未在漏洞描述中出现的文件路径的数量
	Same File Num	提交与漏洞描述中相同文件的数量
	Same File Ratio	相同文件的数量与提交中修改的文件总数的比值
	Unrelated File Num	在提交中出现但未在漏洞描述中出现的文件的数量

	Same Function Num	提交与漏洞描述中相同函数的数量
	Same Function Ratio	相同函数的数量与提交中修改的函数总数的比值
	Unrelated Function Num	在提交中出现但未在漏洞描述中出现的函数的数量
Token 特征	TF-IDF Similarity	提交信息与漏洞描述的 TF-IDF 向量的余弦相似度
	Same Msg_CWE_Token Num	提交信息描述与漏洞 CWE 类型中相同 token 的数量
	Same Msg_CWE_Token Ratio	相同 token 的数量与漏洞 CWE 类型 token 总数的比值
	Same Msg_Desc_Token Num	提交信息描述与漏洞描述中相同 token 的数量
	Same Msg_Desc_Token Ratio	相同 token 的数量与漏洞描述 token 总数的比值
	Max of Same Msg_Desc_Token Frequency	相同 token 中的最大词频
	Sum of Same Msg_Desc_Token Frequency	相同 token 的词频总和
	Average of Same Msg_Desc_Token Frequency	相同 token 的词频均值
	Variance of Same Msg_Desc_Token Frequency	相同 token 的词频方差
	Same Code_Desc_Token Num	提交差异代码与漏洞描述中相同 token 的数量
	Same Code_Desc_Token Ratio	相同 token 的数量与漏洞描述 token 总数的比值
	Max of Same Code_Desc_Token Frequency	相同 token 中的最大词频
	Sum of Same Code_Desc_Token Frequency	相同 token 的词频总和
	Average of Same Code_Desc_Token Frequency	相同 token 的词频均值
	Variance of Same Code_Desc_Token Frequency	相同 token 的词频方差

3.3.1 差异代码特征

现有研究表明，与实现软件功能的提交相比，漏洞对应的安全补丁中修改的代码行数通常会更少[12]。分别统计提交的差异代码中以 “+”和 “-”为起始字符的行的数量，修改行的数量为添加行和删除行的总和。

3.3.2 身份特征

考虑与漏洞和提交相关的三种标识符：CVE ID、Bug ID 和 Issue ID。漏洞有唯一的 CVE ID，在其 NVD 页面中可能会包含指向该漏洞被分配 CVE ID 前所使用的 Bug/Issue ID 详情的超链接。开发人员在进行提交时，可能会在其描述中添加该提交对应修复的 CVE/Bug/Issue ID，可以使用正则表达式从描述中识别这些标识符。包含上述标识符的提交更可能是安全补丁，而非实现功能需求。特别是当漏洞的标识符与提交的标识符能够匹配时，该提交为漏洞对应的安全补丁的可能性非常大，但也不排除提交中提到标识符是为了区分不同的提交，而不是说明该提交的相关信息。

另一方面，Tan 等人对 500 个漏洞以及对应的安全补丁进行分析，得到了 16 种漏洞类型和 15 种漏洞影响[7]。可以从漏洞描述和提交信息描述中识别漏洞类型和漏洞影响，进而对漏洞和补丁的相似性进行判断。具体而言：如果提交中提到的漏洞类型与当前漏洞的类型相同或存在包含关系 (例如溢出和缓冲区溢出)；或

者提交中提到的漏洞类型与当前漏洞的影响存在因果关系 (例如发生缓冲区溢出后可能会出现拒绝服务)；或者反之，当前漏洞的类型与提交中提到的漏洞影响存在因果关系，说明该提交很可能是该漏洞的安全补丁。

3.3.3 时间特征

漏洞的 CVE ID 分配时间会记录在相应 CVE 页面中，提交的时间会记录在相应提交记录中。Tan 等人对 225 个漏洞的 CVE ID 分配时间与安全补丁提交时间进行分析，发现两者的时间间隔往往并不远[7]。该项工作表明，距离漏洞披露时间更近的提交，相较于那些间隔时间更远的提交，更有可能是安全补丁。

3.3.4 位置特征

漏洞描述中可能会提到漏洞所在的文件名或函数名。对应的安全补丁对漏洞进行修复所产生的差异代码中将会出现上述文件名和函数名。以漏洞 CVE-2016-2176 为例，其真实安全补丁所修复的函数为 crypto/x509/x509_obj.c 文件中的 X509_NAME_oneline，而该漏洞描述中提到了同一文件里的同一函数，如图 2 和图 3 所示。这一例子说明，位置特征可以有效地用于揭示漏洞和补丁间的链接。

使用正则表达式识别漏洞描述中的文件路径、文件名和函数名，结合差异代码的结构识别提交中修改的文件名、函数名以及文件所在的路径。计算在提交和漏洞描述中相同文件路径/文件名/函数名的数量和比例，值越大表示提交和漏洞描述在修改内容上相似度越高。此外，还将同时计算在提交中出现但未在漏洞描述中出现的文件路径/文件名/函数名的数量，值越大表示提交和漏洞描述在修改内容上相似度越低。

图 2 CVE-2016-2176 漏洞描述

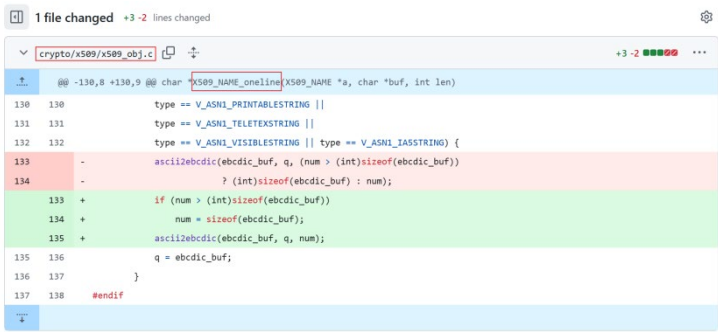

图 3 CVE-2016-2176 对应安全补丁的差异代码

3.3.5 Token 特征

提交信息包括提交的描述和差异代码 (含修改的文件路径、文件名、函数名以及具体代码)。将漏洞描述或提交信息视为单篇文档，所有文档共同构成文档库。文档库 D 中文档 d 的 token t 的 TF-IDF 值计算方式如公式(2)所示，可以通过计算漏洞描述与提交信息的 TF-IDF 向量的余弦相似度来判断两项文本的相似性，如公式(3)所示，其中 $\overrightarrow{d_i}$ 是第 i 个漏洞描述的 TF-IDF 向量，$\vec{c_j}$ 是第 i 个漏洞对应的提交集合 C_i 中的第 j 个提交信息的 TF-IDF 向量。在提交集合中与漏洞描述更相似的提交将会获得更高的 TF-IDF 相似度得分。

$$\mathrm{Cosine}\left(\vec{d_i}, \vec{c_j}\right) = \frac{\vec{d_i} \cdot \vec{c_j}}{|\vec{d_i}| \times |\vec{c_j}|}. \tag{3}$$

另外，亦从提交信息描述与漏洞 CWE 类型、提交信息描述与漏洞描述、提交信息差异代码与漏洞描述三个角度计算公共词的数量和比例，作为一类特征。考虑到在一对信息中，可能出现多个公共词，因此将计算公共词词频的最大值、和、均值和方差等统计特征。

3.4 语义特征生成

由于漏洞补丁相似性度量任务涉及到漏洞描述和提交信息描述组成的自然语言对，选择 RoBERTa 预训练模型来生成文本向量，以学习文本中丰富的语义信息，因为 RoBERTa 在处理自然语言对的任务上具有良好的性能，且能比其他流行的预训练模型在自然语言分类方面取得更好的效果[21]。图 4 展示了 MFLocator 中语义特征生成模块的结构。该模块分为两个阶段：对 RoBERTa 模型的微调和特征生成。

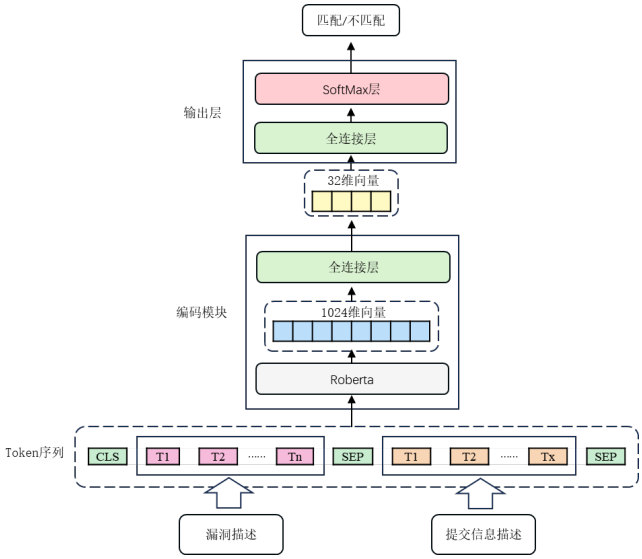

图 4 语义特征生成示意图

- 对 RoBERTa 模型的微调：

对训练集进行正负样本比例为 1:N 的随机负采样，即对于每一个漏洞，随机取其对应的 label 为 0 的 N 个提交 ($N = 1, 10, 50$)，与 label 为 1 的提交组成专用于微调的训练集。为了实现对 RoBERTa 模型的微调，使用负采样后的训练集，将漏洞与提交是否匹配作为训练目标，并使用交叉熵损失函数 (cross-entropy loss function) 来缩小预测值和真实值之间的差异。

微调过程中，将训练集里漏洞描述文本的 token 序列和提交信息描述文本的的 token 序列串联，得到如公式(4)所示格式的 token 序列：

$$[\mathrm{CLS}], i_1, i_2, \ldots i_k, \ldots, i_m, [\mathrm{SEP}], c_1, c_2, \ldots, c_k, \ldots, c_n, [\mathrm{SEP}], \tag{4}$$

其中 i_k 和 c_k 分别代表漏洞描述文本和提交信息描述文本中的第 k 个 token，[CLS]表示这是一个用于分类的向量，[SEP]表示分隔符。为了保证输入长度一致，在对两个序列进行合并前，还需要对上述序列进行头部截断

(head-only truncation)。具体而言，为 token 序列设置长度最大值*max_sequence_length* (本实验中为 512)，当漏洞描述文本或提交信息描述文本的 token 序列长度小于*max_sequence_length*的 1/2 时，将使用特殊的空格 [PAD] 作为填充插入到序列中；超出时则会从末尾将其截断。将该序列输入进 RoBERTa 模型，得到一个 1024 维的向量。由于这对本文所使用的分类器来说过长，因此使用全连接层将该向量压缩成一个 32 维向量，并进一步将其传递到输出层 (由一个全连接层和一个 Softmax 层组成)进行压缩，最终归一化为预测输出类的概率分布。

- 特征生成：

RoBERTa 微调完成后，即可用于 MFLocator 的特征生成任务。具体而言，将待表征的一对漏洞描述文本和提交信息描述文本处理成如公式(4)所示的 token 序列，输入进微调好的 RoBERTa 模型，得到如图 4 中所示的 32 维向量作为语义特征。

3.5 特征拼接

在特征拼接阶段，将第 3.4 节输出的语义特征与第 3.3 节生成的手工特征进行拼接操作，得到维数是二者之和的特征。

3.6 排序综合

在训练环节，选择在不平衡数据上表现较好的 XGBoost、LightBGM 和 CatBoost 模型作为分类器。在使用拼接特征对模型进行训练后，每个分类器都可以对漏洞和提交进行匹配性预测。使用基于投票的方法综合三个分类器的结果，得出最终的排序。具体而言，对于每个漏洞，使用三个分类器预测得到的匹配性分值，分别对候选提交进行排序。对于同一个候选提交在不同分类器中的排名，选择最接近的两个排名，根据二者的均值确定其最终排序，得到综合排序结果。这种方法可以减少单一模型可能存在的偏差或错误，从而提高整体排序的准确性[12]。

4 实验设置

4.1 研究问题 (Research Question, RQ)

- RQ1：微调中正负样本比例影响分析

本 RQ 对语义特征提取阶段中微调部分所使用训练集的负采样比例进行研究，分析正负样本比例对语义特征提取效果的影响。对于每一个漏洞，随机取其对应的 label 为 0 的 N 个提交 (N=1, 10, 50)，与 label 为 1 的提交组成新的正负样本比例为 1:N 的训练集，得到 MFLocator 方法的三种变体：$\text{MFLocator}_{1:1}$，$\text{MFLocator}_{1:10}$ 和 $\text{MFLocator}_{1:50}$。分析三种变体的 Recall@K (K=1, 2, ..., 10)、Manual Efforts@K (K=1, 2, ..., 10)和 NDCG@K (K=1, 5)指标值。

- RQ2：MFLocator有效性分析

本 RQ 将 VCMatch 这一漏洞补丁定位领域当前代表性技术作为比较对象，分析 MFLocator 方法的有效性。使用两种方法在同一数据集上进行补丁定位，分析两种方法的 Recall@K (K=1, 2, ..., 10)、Manual Efforts@K (K=1, 2, ..., 10)和 NDCG@K (K=1, 5)指标值。

- RQ3：手工特征和语义特征重要性分析

本 RQ 进行消融实验，分析 MFLocator 方法中手工特征和语义特征对补丁定位效果的重要性。将 MFLocator 方法形成两种变体：消融掉手工特征，只使用语义特征形成 MFLocator_{h-}；以及消融掉语义特征，只使用手工特征的 MFLocator_{s-}。将两种变体方法的 Recall@K (K=1, 2, ..., 10)、Manual Efforts@K (K=1, 2, ..., 10)和 NDCG@K (K=1, 5)指标值同 MFLocator 方法进行比较。

4.2 数据集

由于在大赛官网提供的数据集中发现了标签错误等问题，**经与主办方沟通并获得同意，本实验使用一套已在软件工程及安全领域得到认可的数据集**[11, 12]。该数据集共计 1669 条漏洞，包含漏洞 CVE ID、漏洞出现的代码存储库以及对应安全补丁的提交哈希值等，如表 2 所示。

表 2 各代码存储库的漏洞数量

代码存储库	漏洞数量
Linux	286
php-src	153
phpMyadmin	95
FFmpeg	215
Moodle	119
QEMU	141
Wireshark	304
Jenkins	108
OpenSSL	92
ImageMagick	156

对于漏洞 v_i，从 CVE 页面中获取漏洞的 CVE ID 分配时间$time_i$，从 NVD 页面上获取漏洞的描述信息$desc_i$、CWE 类型cwe_i和相关超链接$hyperlinks_i$。选择对应的安全补丁作为正样本，从相应的代码存储库中寻找能够计算差异代码的 149 条非安全补丁提交作为负样本，组成漏洞 v_i 的提交集合C_i。对于C_i中的提交c_j，使用 GitPython 库获取提交的描述信息、差异代码和提交时间。在实际操作过程中，Li 等人[11]、Pan 等人[24]和 Le 等人[25]均对提交中过长的差异代码进行截断。参考前人的做法并出于实验成本的考虑，本实验中仅保留差异代码的前 1000 行。

4.3 评估指标

MFLocator 将漏洞定位到补丁任务视为排序任务，因此使用 Recall@K、Manual EffortsE@K 和 NDCG@K 这三种在该领域已经得到广泛应用的指标进行性能评估。

Recall@K 表示真实安全补丁被排序在 Top-K 位置的情况占所有安全补丁的比值。值越高表示越多的安全补丁被排序在 Top-K 内，需要开发人员在 Top-K 外寻找安全补丁的漏洞数量越少，模型性能也越好。

Manual Efforts@K 表示在 Top-K 中寻找安全补丁所需要的检查次数。值越低表示开发人员根据排序结果定位到安全补丁的工作量越小，模型性能越好。该指标的计算如公式(5)所示，其中 N 为安全补丁总数，$rank_i$ 为第 i 个安全补丁的排名。

$$\text{Manual Efforts@}K = \frac{\sum_{i=1}^{N} \min(rank_i, K)}{N}. \tag{5}$$

NDCG@K 是折损累计增益 DCG@K 与理想折损累计增益 iDCG@K 的比值，如公式(6)所示。DCG@K 考虑 Top-K 中安全补丁的排名情况，如公式(7)所示，其中 $r(i)$是排名第 i 的提交与漏洞信息的相关性分数，如公式(8)所示。iDCG@K 为理想情况 (即假设安全补丁排在首位)下的 DCG@K 值。NDCG 值越高，表示模型将安全补丁放在更靠前的位置，说明模型性能越好。将所有漏洞对应提交排序的 NDCG 值取平均，作为最终的指标值。

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{iDCG@}K}, \tag{6}$$

$$\mathrm{DCG@}K=\sum_{i=1}^{K}\frac{2^{r(i)}-1}{\log_2(1+i)}, \tag{7}$$

$$r(i)=\begin{cases}1, & \text{如果代码提交是安全补丁}\\ 0. & \text{如果代码提交不是安全补丁}\end{cases} \tag{8}$$

下面以一个例子演示 NDCG 指标的计算过程。假设在一个排序过程中，排名前 5 的结果的相关性分数分别为 0, 1, 0, 0 ,0，它们的折损增益值 DCG 如表 3 所示，根据公式(7)可得 DCG@5 的值为 0.6309。而 iDCG 将它们按照真实的相关性分数进行排序 (即令安全补丁排名第一)，所得到的折损增益值如表 4 所示，可得 iDCG@5 的值为 1。根据公式(6)将 DCG@5 与 iDCG@5 的值相除，得到 NDCG@5 的值为 0.6309。

表 3 DCG@5 值示例

排名i	标签	相关性分数$r(i)$	折损增益$\frac{2^{r(i)}-1}{\log_2(1+i)}$
1	0	0	0
2	1	1	0.6309
3	0	0	0
4	0	0	0
5	0	0	0

表 4 iDCG@5 值示例

排名i	标签	相关性分数$r(i)$	折损增益$\frac{2^{r(i)}-1}{\log_2(1+i)}$
1	1	1	1
2	0	0	0
3	0	0	0
4	0	0	0
5	0	0	0

4.4 实验硬件

RoBERTa 模型微调时，设置*batchsize*为 8，训练轮数为 10 轮，使用 AdamW 优化器，学习率为 1e-5；语义特征生成时，设置*batchsize*为 16。实验在 Ubuntu 20.04 64 位机器 (60GB 内存，4 个 Intel Xeon Gold 5222 CPU 处理器，2 块 NVIDIA RTX A6000 GPU)上进行。实验采用五折交叉验证的方式在整个数据集上评估模型的性能。

5 结果与讨论

5.1 RQ1：微调中正负样本比例影响分析

对于 $\mathrm{MFLocator}_{1:1}$，$\mathrm{MFLocator}_{1:10}$ 和 $\mathrm{MFLocator}_{1:50}$ 三种变体，实验结果的 Recall@K (K=1, 2, ..., 10)、Manual Efforts@K (K=1, 2, ..., 10)和 NDCG@K (K=1, 5)指标值分别如表 5、表 6 和表 7 所示。可以看出，当 K 取最小值 1，即正负样本平衡时，MFLocator 取得了最高的 Recall 和 NDCG 指标值以及最小的 Manual Efforts 指标值；而随着 K 取值的增大，MFLocator 取得的 Recall 和 NDCG 指标值逐渐减小，Manual Efforts 指标值逐渐增大(即均朝着有效性下降的方向发展)。这一结果表明，微调中正负样本的平衡能够帮助增大 Recall 和 NDCG，降低 Manual Efforts，提升模型的整体性能。因此，后续的实验均在语义特征提取的微调阶段使用正负样本比例为 1:1 的负采样，即**下文中的 MFLocator 均指 $\mathrm{MFLocator}_{1:1}$**。

表 5 微调时不同负样本比例下方法的 Recall@*K* 指标值

Recall@	1	2	3	4	5	6	7	8	9	10
$MFLocator_{1:1}$	**96.04%**	**98.20%**	**98.74%**	**99.10%**	**99.16%**	**99.22%**	**99.22%**	**99.22%**	**99.34%**	**99.40%**
$MFLocator_{1:10}$	95.14%	97.06%	97.66%	98.38%	98.56%	98.80%	98.92%	99.10%	99.10%	99.10%
$MFLocator_{1:50}$	93.58%	96.40%	97.36%	98.14%	98.38%	98.68%	98.74%	98.80%	98.80%	98.92%

表 6 微调时不同负样本比例下方法的 Manual Efforts@*K* 指标值

Manual Efforts@	1	2	3	4	5	6	7	8	9	10
$MFLocator_{1:1}$	1	**1.0395**	**1.0575**	**1.0701**	**1.0790**	**1.0874**	**1.0952**	**1.1030**	**1.1108**	**1.1174**
$MFLocator_{1:10}$	1	1.0485	1.0778	1.1012	1.1174	1.1318	1.1437	1.1545	1.1635	1.1725
$MFLocator_{1:50}$	1	1.0641	1.1000	1.1264	1.1449	1.1611	1.1743	1.1869	1.1989	1.2109

表 7 微调时不同负样本比例下方法的 NDCG@*K* 指标值

NDCG@	1	5
$MFLocator_{1:1}$	**0.9604**	**0.9785**
$MFLocator_{1:10}$	0.9514	0.9703
$MFLocator_{1:50}$	0.9358	0.9627

5.2 RQ2：MFLocator有效性分析

表 8、表 9 和表 10 分别展示了 MFLocator 方法所取得的 Recall@*K* (*K*=1, 2, ..., 10)、Manual Efforts@*K* (*K*=1, 2, ..., 10)和 NDCG@*K* (*K*=1, 5)指标值。上述数据可以用图 5、图 6 和图 7 更直观地表示，其中横坐标表示 *K* 的值，纵坐标表示对应的指标值。

从表 8 可以看到，相比 VCMatch 方法，MFLocator 方法在 Recall 上有提升，尤其是在 *K* 较小时。具体而言，当 *K*=1 较小时，MFLocator 方法相较于 VCMatch 方法在 Recall 上提升了约 3%。而当 *K* 较大时，尽管二者指标值的差距有所减小，但 MFLocator 方法的 Recall 值仍然优于 VCMatch。

从表 9 可以看到，MFLocator 方法的 Manual Efforts 值相较于 VCMatch 方法实现了提升。具体而言，VCMatch 方法在 *K* 较小时，相较于 MFLocator 方法的提升幅度 Manual Efforts 较小；而随着 *K* 的增大，MFLocator 的提升幅度逐渐提升，直到 *K*=10 时达到与 VCMatch 的最大差值 0.1228。

从表 10 可以看到，相比 VCMatch 方法，MFLocator 方法在 NDCG 指标上同样实现了提升。具体而言，当 *K*=1 和 *K*=5 时，MFLocator 方法相较于 VCMatch 方法在 NDCG 上的提升幅度分别为 3.16%和 2.07%。

综上所述，MFLocator 方法相较于 VCMatch 方法，能够提升 Recall 和 NDCG 指标值，同时降低手工开销 Manual Efforts，这意味着 MFLocator 可以实现更加有效的漏洞定位到补丁，同时降低人类程序员的手工检查成本，实现更好的漏洞补丁定位效果。

表 8 MFLocator 的 Recall@*K* 指标值

Recall@	1	2	3	4	5	6	7	8	9	10
VCMatch	93.10%	96.10%	96.76%	97.60%	98.08%	98.20%	98.50%	98.50%	98.80%	98.86%
MFLocator	**96.04%**	**98.20%**	**98.74%**	**99.10%**	**99.16%**	**99.22%**	**99.22%**	**99.22%**	**99.34%**	**99.40%**

表 9 MFLocator 的 Manual Efforts@*K* 指标值

Manual Effort@	1	2	3	4	5	6	7	8	9	10
VCMatch	1	1.0665	1.1066	1.1360	1.1587	1.1785	1.1971	1.2121	1.2264	1.2402
MFLocator	1	**1.0395**	**1.0575**	**1.0701**	**1.0790**	**1.0874**	**1.0952**	**1.1030**	**1.1108**	**1.1174**

表 10 MFLocator 的 NDCG@*K* 指标值

NDCG@	1	5
VCMatch	0.9310	0.9587
MFLocator	**0.9604**	**0.9785**

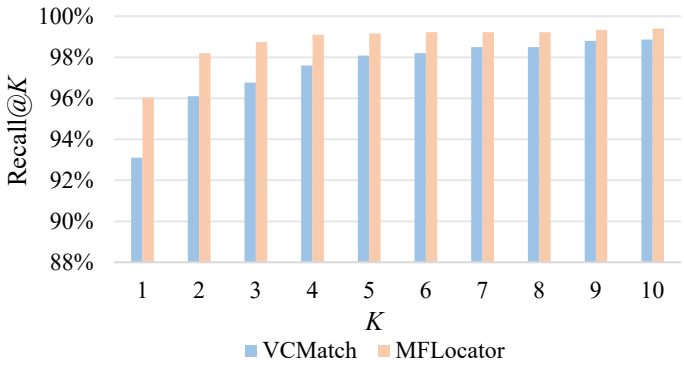

图 5 MFLocator 的 Recall@*K* 指标值

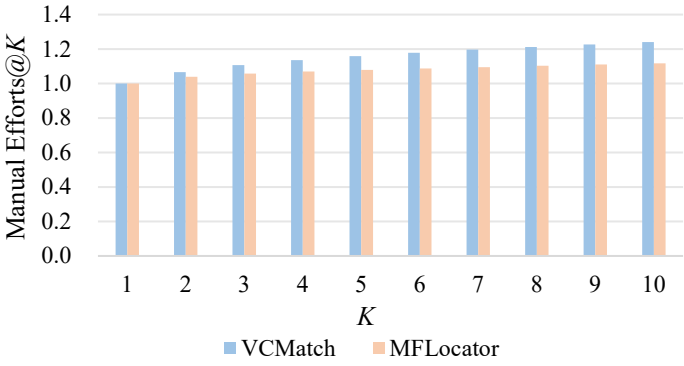

图 6 MFLocator 的 Manual Efforts@*K* 指标值

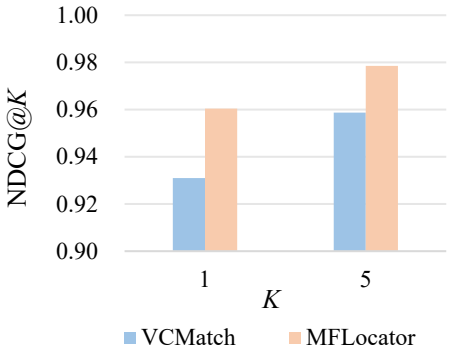

图 7 MFLocator 的 NDCG@*K* 指标值

5.3 RQ3：手工特征和语义特征重要性分析

表 11、表 12 和表 13 分别展示了在消融实验中，MFLocator 方法两种变体所取得的 Recall@*K* (*K*=1, 2, ..., 10)、Manual Efforts@*K* (*K*=1, 2, ..., 10)和 NDCG@*K* (*K*=1, 5)指标值。

从表 11 可以看到，就 Recall 指标 (值越高效果越好)而言，消融掉手工特征和语义特征中的任何一个，都会造成 MFLocator 方法效果的下降。具体而言，$\text{MFLocator}_{h\text{-}}$相较于 MFLocator 的下降幅度在 9.53%到 17.73%之间，$\text{MFLocator}_{s\text{-}}$相较于 MFLocator 的下降幅度在 0.30%到 2.70%之间。这表明，手工特征和语义特征均对方法的效果起到了正向的增益作用。值得留意的是，上述最大的下降幅度均是在 *K*=1 时取得的，这表明两种特征在 *K*=1 时均对方法具有更大的增益，这比 *K* 值较大时的降幅具有更重要的参考意义，因为开发者往往更聚焦于排名靠前的那些补丁。

从表 12 可以看到，就 Manual Efforts 指标 (值越低效果越好)而言，消融掉手工特征和语义特征中的任何一个，都会造成 MFLocator 方法效果的下降。具体而言，$\text{MFLocator}_{h\text{-}}$相较于 MFLocator，Manual Efforts 指标的增加值在 0.1773 到 1.0785 之间，MFLocator_{s}指标的增加值在 0.0270 到 0.1030 之间。上述结果表明，手工特征和语义特征均对方法的效果起到了正向的增益作用。

与上述两个指标类似，从表 13 同样可以观察到，就 NDCG 指标 (值越高效果越好)而言，消融掉手工特征和语义特征中的任何一个，都会造成 MFLocator 方法效果的下降。具体而言，当 *K*=1 和 *K*=5 时，$\text{MFLocator}_{h\text{-}}$相较于 MFLocator 的下降分别为 0.1773 和 0.1407，$\text{MFLocator}_{s\text{-}}$相较于 MFLocator 的下降分别为 0.0287 和 0.0208，表明手工特征和语义特征均对方法的效果起到了正向的增益作用。

此外，从数据中还可以发现，相比只使用语义特征，只使用手工特征在 Recall、Manual Efforts 和 NDCG 维度上均表现出更好的效果，但仍然差于同时使用两种特征的原方法。这意味着手工特征和语义特征对有效定位补丁表现出不同的重要性，二者同时使用可以实现互补的效果。

表 11 MFLocator 及两种变体的 Recall@*K* 指标值

Recall@	1	2	3	4	5	6	7	8	9	10
$\text{MFLocator}_{h\text{-}}$	78.31%	83.70%	86.21%	87.65%	88.13%	88.61%	89.03%	89.27%	89.45%	89.87%
$\text{MFLocator}_{s\text{-}}$	93.34%	96.22%	96.88%	97.84%	98.50%	98.62%	98.74%	98.80%	98.98%	99.10%
MFLocator	**96.04%**	**98.20%**	**98.74%**	**99.10%**	**99.16%**	**99.22%**	**99.22%**	**99.22%**	**99.34%**	**99.40%**

表 12 MFLocator 及两种变体的 Manual Efforts@*K* 指标值

Manual Efforts@	1	2	3	4	5	6	7	8	9	10
$\text{MFLocator}_{h\text{-}}$	1	1.2168	1.3798	1.5176	1.6411	1.7579	1.8735	1.9832	2.0904	2.1959
$\text{MFLocator}_{s\text{-}}$	1	1.0665	1.1042	1.1354	1.1569	1.1719	1.1875	1.1983	1.2103	1.2204
MFLocator	1	**1.0395**	**1.0575**	**1.0701**	**1.0790**	**1.0874**	**1.0952**	**1.1030**	**1.1108**	**1.1174**

表 13 MFLocator 及两种变体的 NDCG@*K* 指标值

NDCG@	1	5
$\text{MFLocator}_{h\text{-}}$	0.7831	0.8378
$\text{MFLocator}_{s\text{-}}$	0.9317	0.9577
MFLocator	**0.9604**	**0.9785**

6 讨论

对实验结果进行回顾，发现有 10 个漏洞对应的安全补丁排在了 Top-10 之外。对这 10 个定位效果不佳的漏洞进行手工分析，发现了两种主要的原因：

- 漏洞描述信息不够详细

以漏洞 CVE-2015-7536 为例，其真实补丁的预测排名为 148。图 8 展示了 CVE-2015-7536 在 NVD 中的漏洞描述信息，可以看到该漏洞描述仅包含了一句概括性的信息，而没有详细地介绍漏洞出现原因、出现位置等重要信息，模型难以从这样简略的信息中提取到有效的语义信息，因此可能造成匹配效果不佳。该类情况在 10 个漏洞中占 5 个。

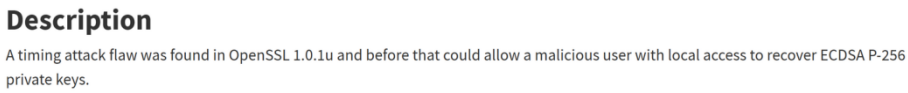

图 8　CVE-2015-7536 在 NVD 中的漏洞描述

- 提交信息描述不够详细

以漏洞 CVE-2000-1254 为例，其真实补丁的预测排名为 146。图 9 展示了 CVE-2000-1254 真实补丁的提交内容。可以看到其标题部分只有“Bug fix for 64 bit HP-UX”这一概括性信息，下方的具体内容部分也只包含提交作者等与漏洞种类和描述无关的信息。这些信息很难对补丁定位任务起到帮助，因此可能造成匹配效果不佳。该类情况在 10 个漏洞中占 3 个。

图 9　CVE-2000-1254 对应的补丁提交

7　总结与展望

针对大量开源软件漏洞未与其安全补丁建立链接的问题，研究并实现了一种基于多源特征融合的开源软件漏洞定位到补丁方法 MFLocator。从五个维度分析漏洞与提交的相关性并生成相应的手工特征，同时使用预训练模型提取由漏洞描述和提交信息描述组成的自然语言对中的语义特征。将两组特征拼接后输入到三个分类模型，得到漏洞与提交的匹配概率分数。最后使用基于投票的方式，选择三个模型的排序中最相近的两个取均值得到最终的排序。在实验中，首先研究语义特征生成的微调阶段中，训练集正负样本比例对补丁定位效果的影响，实验表明使用平衡的数据集进行微调可以取得更佳的效果；然后通过与该领域现有的代表性方法进行比较，在 Recall、Manual Efforts 和 NDCG 指标维度证实了 MFLocator 方法的有效性；最后进行消融实验，分别研究手工特征和语义特征在 MFLocator 方法中的重要性，实验结果表明二者同时使用时，补丁定位的效果最好，证实二者对于补丁定位任务可以起到不同角度的贡献。此外，还讨论了导致不佳定位结果的两个可能原因，即漏洞描述和提交信息描述不够详细。

在后续的研究中，拟利用漏洞描述和提交信息中提到的超链接对定位源信息进行补充，使特征更加多元化，并计划在更大规模的数据集上开展实验，进一步验证 MFLocator 方法的有效性和泛化能力，以期更好地建立漏洞与其安全补丁间的链接。

参考文献

[1] 杨云帆. 面向开源软件漏洞的补丁管理技术研究及平台实现[D]. 扬州大学. 2023.

[2] 纪守领，王琴应，陈安莹，等. 开源软件供应链安全研究综述[J]. 软件学报. 2023，34(03)：1330-1364.

[3] Liu C, Chen S, Fan L, et al. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem[C]. Proceedings of the 44th International Conference on Software Engineering. 2022: 672-684.

[4] Common Vulnerabilities and Exposures[EB/OL]. 2024. https://cve.mitre.org/.

[5] National Vulnerability Database[EB/OL]. 2024. https://nvd.nist.gov/.

[6] Guo H, Chen S, Xing Z, et al. Detecting and augmenting missing key aspects in vulnerability descriptions[J]. ACM Transactions on Software Engineering and Methodology, 2022. 31(3): 1-27.

[7] Tan X, Zhang Y, Mi C, et al. Locating the security patches for disclosed oss vulnerabilities with vulnerability-commit correlation ranking[C]. Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. 2021: 3282-3299.

[8] Everson D, Cheng L, Zhang Z. Log4shell: Redefining the web attack surface[C]. Proceedings of the Workshop Meas., Attacks, Defenses Web. 2022: 1-8.

[9] 2024 中国软件供应链安全分析报告[EB/OL]. 2024. https://www.qianxin.com/threat/reportdetail?report_id=316.

[10] Xu C, Chen B, Lu C, et al. Tracking patches for open source software vulnerabilities[C]. Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 860-871.

[11] Li K, Zhang J, Chen S, et al. PatchFinder: A Two-Phase Approach to Security Patch Tracing for Disclosed Vulnerabilities in Open-Source Software[C]. Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2024: 590-602.

[12] Wang S, Zhang Y, Bao L, et al. VCMatch: a ranking-based approach for automatic security patches localization for OSS vulnerabilities[C]. 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering. 2022: 589-600.

[13] Hogan K, Warford N, Morrison R, et al. The challenges of labeling vulnerability-contributing commits[C]. 2019 IEEE International Symposium on Software Reliability Engineering Workshops. 2019: 270-275.

[14] Liu Y, Ott M, Goyal N, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv Preprint arXiv: 1907.11692. 2019: 1-13.

[15] Devlin J, Chang MW, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv Preprint arXiv: 1810.04805. 2018: 1-16.

[16] Luiz F C, de Oliveira Rodrigues B R, Parreiras F S. Machine learning techniques for code smells detection: An empirical experiment on a highly imbalanced setup[C]. Proceedings of the XV Brazilian Symposium on Information Systems. 2019: 1-8.

[17] Chen T and Guestrin C. Xgboost: A scalable tree boosting system[C]. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016: 785-794.

[18] Ke G, Meng Q, Finley T, et al. LightGBM: a highly efficient gradient boosting decision tree[C]. Proceedings of the 31st International Conference on Neural Information Processing Systems. 2017: 3149–3157.

[19] Prokhorenkova L, Gusev G, Vorobev A, et al. CatBoost: unbiased boosting with categorical features[C]. Proceedings of the 32nd International Conference on Neural Information Processing Systems. 2018: 6639-6649.

[20] Wang S, Dong P and Tian Y. A novel method of statistical line loss estimation for distribution feeders based on feeder cluster and modified XGBoost[J]. Energies. 2017, 10(12): 2067.

[21] Lan J, Gong L, Zhang J, et al. BTLink: automatic link recovery between issues and commits based on pre-trained BERT model[J]. Empirical Software Engineering. 2023, 28(4): 103.

[22] Loper E and Bird S. Nltk: The natural language toolkit. arXiv Preprint arXiv: cs/0205028. 2002.

[23] SSalton G, Yu C T. On the construction of effective vocabularies for information retrieval[J]. Acm Sigplan Notices. 1973, 10(1): 48-60.

[24] Pan S, Bao L, Xia X, et al. Fine-grained commit-level vulnerability type prediction by CWE tree structure[C]. Proceedings of the 45th International Conference on Software Engineering. 2023: 957-969.

[25] Le T H M, Hin D, Croft R, et al. Deepcva: Automated commit-level vulnerability assessment with deep multi-task learning[C]. Proceedings of the 36th International Conference on Automated Software Engineering. 2021: 717-729.